

שלחנו סוכן להזמין חופשה ואז התערבתם

סוכנים, דפדפנים, Jev ו-AWS

תומר גל · tomerwave.com

האתר והמלונות בדויים. אין רכישה אמיתית.



תומר גל

קצת על מי שנתן לסוכן דפדפן

תשתיות דפדפן לסוכני AI

בניתי תשתיות דפדפן וניהול סשנים מאומתים

אמינות סוכני AI

הובלתי פיתוח, כולל תיקון תקלות בזמן ריצה

מחוץ למחשב

טיפוס וריקוד. גם שם כדאי לדעת מה הצעד הבא

tomergal.yaml

```
1  tomer_gal:
2    role: Fractional CTO & Tech Advisor
3    based: Tel Aviv
4    focus: [Architecture, AI agents, Automation]
5    before:
6      - browser_infrastructure: authenticated sessions
7      - agent_reliability: runtime remediation
8    stack:
9      languages: [TypeScript, Python, Java, Kotlin]
10     infra: [AWS, Terraform, Kubernetes]
11    started_coding:
12      age: 12
13    website: tomerwave.com
```

החופשה שלנו

משך

3 לילות

אותם תאריכים בכל המלונות

תקציב

€600

לכל השהייה

תנאי חובה

נגישות

לא העדפה. בלי פשרות.

המטרה: למצוא מלון מתאים ולעצור לפני התשלום.

השליטה בטרמינל

זה נראה חלק מדי?

מה הייתם בודקים לפני שהסוכן ממשיך?

01 תוצאות חיפוש

02 פתיחת מלון

03 בדיקת תנאים

בחירות מתוסרטות. בלי קריאות מודל



בחרתי מלון!

מלון A במחיר €520. עוד לא לחצנו.

אתם משנים את התנאים באותה ריצה:

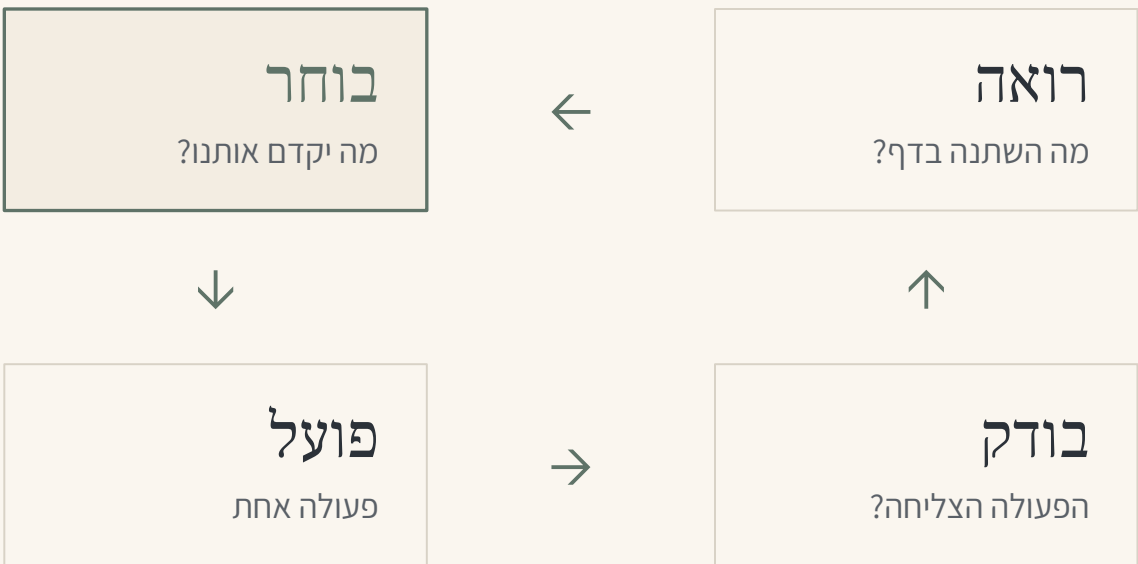
A המלון התמלא

B התקציב ירד ל-€500



מה הופך את זה לסוכן?

הסוכן בוחר את הצעד הבא לפי מה שקרה



מסלול קבוע

הקוד קובע מראש את הסדר. גם if הוא החלטה של הקוד.

לולאת סוכן

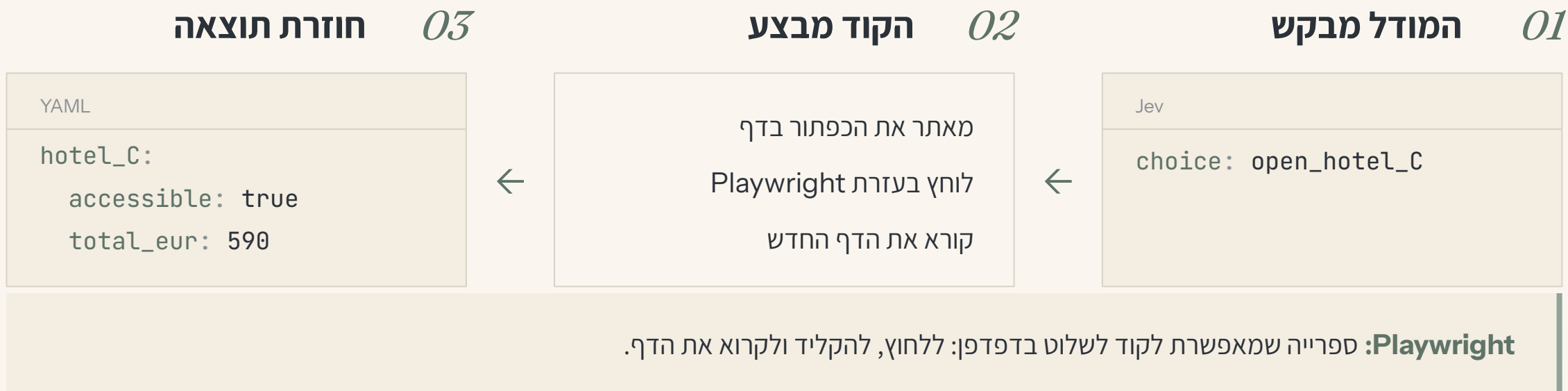
המודל בוחר את הפעולה הבאה לפי המצב החדש.

בדמו שלנו

רואה שמלון A מלא, בוחר לחזור לתוצאות, לוחץ ובודק שנטענו תוצאות.

החידוש: מי בוחר את הפעולה, ולפי איזה מידע. איסוף המידע, ההרשאות והבדיקה נשארים בקוד.

כלי (Tool): פונקציה שהסוכן יכול להפעיל



המודל מבקש. הקוד מבצע ומחזיר תוצאה.

Jev

מודל שבוחר מתוך אפשרויות

כמו `enum`: מזהה אחד מתוך רשימה סגורה, לא פסקה.

מצב קצר + שאלה + אפשרויות

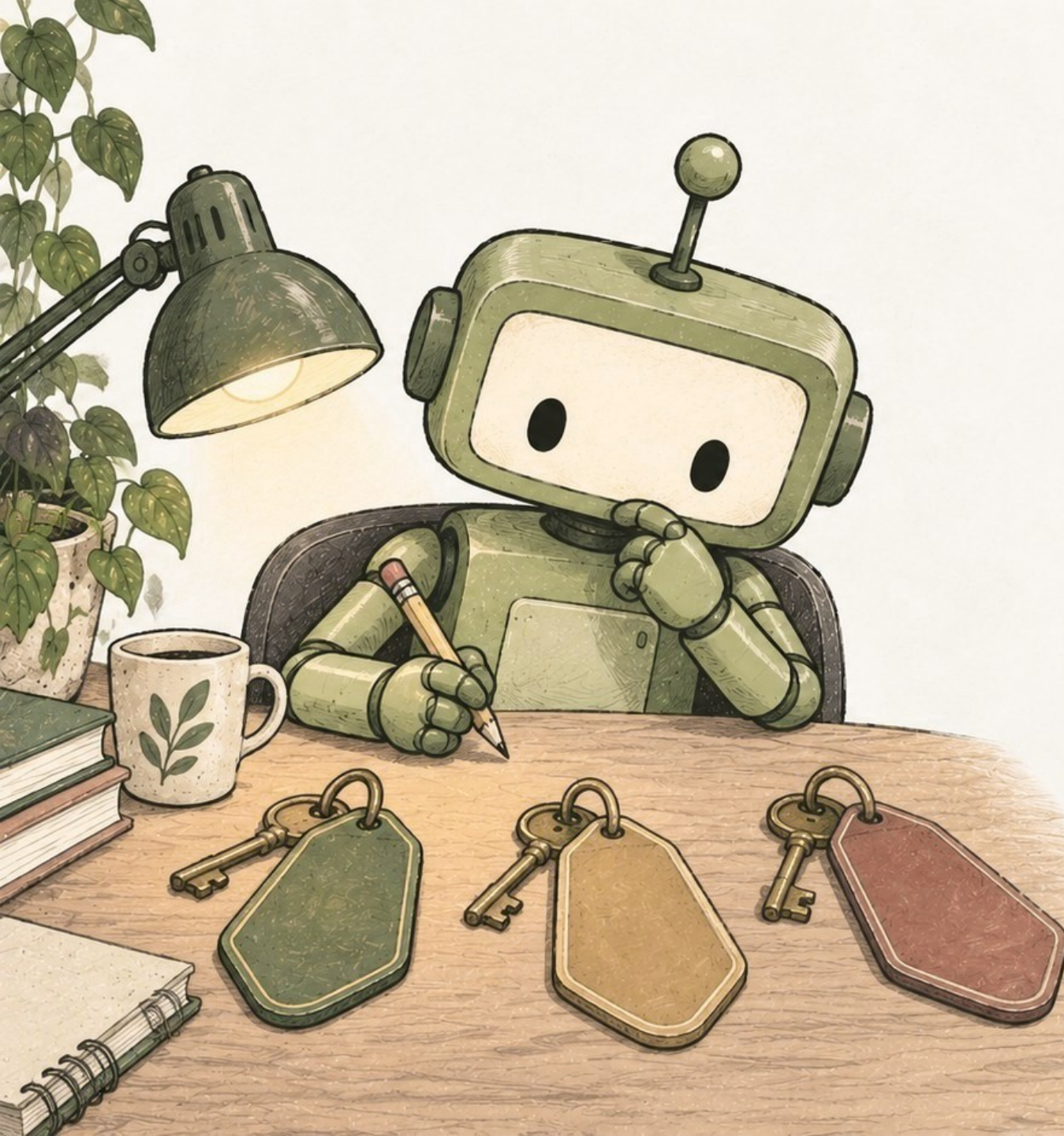
נכנס

בחירה + הסתברויות

יוצא

המטרה נקבעת בקוד. Jev בוחר איזו פעולה עכשיו.

TypeSafe AI · System One



מה Jev מקבל מהקוד שלנו?

YAML

```
1 subgoal: find an accessible hotel
2 budget_eur: 600
3 nights: 3
4 page: hotel_details
5 ruled_out: { A: sold_out }
6 hotel_A: sold_out
7 available_actions: [back_to_results, stop]
```

המצב והפעולות: רק מה שזמין כרגע

←
הקוד מתמצת

vacation-demo.local/hotel/A

מלון A

€520 · 3 לילות · נגיש

אין חדרים פנויים

הזמנה

חזרה לתוצאות

הדף (DOM): כותרות, כפתורים ושדות

Jev לא רואה את הדפדפן. הקוד שלנו מכין לו את המצב ואת הפעולות.



בחירה שאפשר לחבר לקוד

TypeScript

```
1  const result = await client.systemOne({  
2    state: compactState,  
3    questions: {  
4      nextAction: choice(QUESTION, {  
5        back_to_results: "Return to the search results",  
6        stop: "Stop and ask the traveler what to change",  
7      })),  
8    },  
9  });  
10 const answer = result.answers.nextAction;
```

1 המצב

מה קורה עכשיו בדף

2 האפשרויות

רשימה סגורה שהקוד שלנו נותן

3 המזהה שחוזר

stop או back_to_results

לא פסקה.

מזהה מתוך רשימה.

מתוך wave-lab-1 src/jev.ts

בחירה תקינה יכולה להיות שגויה

בדף של מלון B, הפעולות הזמינות: `select_hotel` · `back_to_results` · `stop`

נכונה

מתאימה למטרה

וגם למצב הנוכחי

choice: `select_hotel`

✗ אבל מלון B לא נגיש

תקינה

בחירה מתוך הרשימה

המזהה קיים ובפורמט תקין

choice: `select_hotel`

✓ המזהה קיים ברשימה

גם **confidence** גבוה לא מחליף בדיקה.

ביטחון גבוה אומר שהמודל בטוח, לא שהוא צודק.

איפה AWS בתמונה?



המצב החדש חוזר ל-Jev. בדמו: find_hotel אחד, בלי חזרה אוטומטית למתכנן.

איפה הדפדפן רץ? Playwright הוא הנהג, AgentCore Browser הוא הרכב, ו-Live View מראה לקהל את הדפדפן בזמן אמת.

Jev בוחר. הקוד בודק ומבצע.

המסלול החי אופציונלי ולא נבדק כאן

בין הבחירה ללחיצה

1 העולם השתנה?

ממתינים ומוודאים את גרסת ההחלטה

2 התנאים עדיין מתקיימים?

הבדיקה קוראת את התקציב העדכני

3 רק אז לוחצים

ומיד קוראים את התוצאה

TypeScript

```
1 await presenter.checkpoint("before_click", id);
2 if (decisionRevision  $\neq$  presenter.revision) return;
3
4 const fresh = await readState(browser);
5 const action = registry.get(id);
6 if (!action?.allowed(fresh).ok) return;
7 await action.run(browser);
8 return await readState(browser);
```

loop.ts ו-executor.ts, מקוצר

פעולה משמעותית, כמו הזמנה, נבדקת סופית בצד השרת.



אותה ריצה. ההחלטה הישנה נדחית.

01

השינוי

A התמלא לפני הלחיצה

```
hotel_A: sold_out
```

02

הבדיקה

דחייה לפני לחיצה ←

```
code: world_changed
```

03

סיבוב שני

← **מלון C נגיש ובתקציב**

```
choice: open_hotel_C
hotel_C:
  accessible: true
  total_eur: 590
```

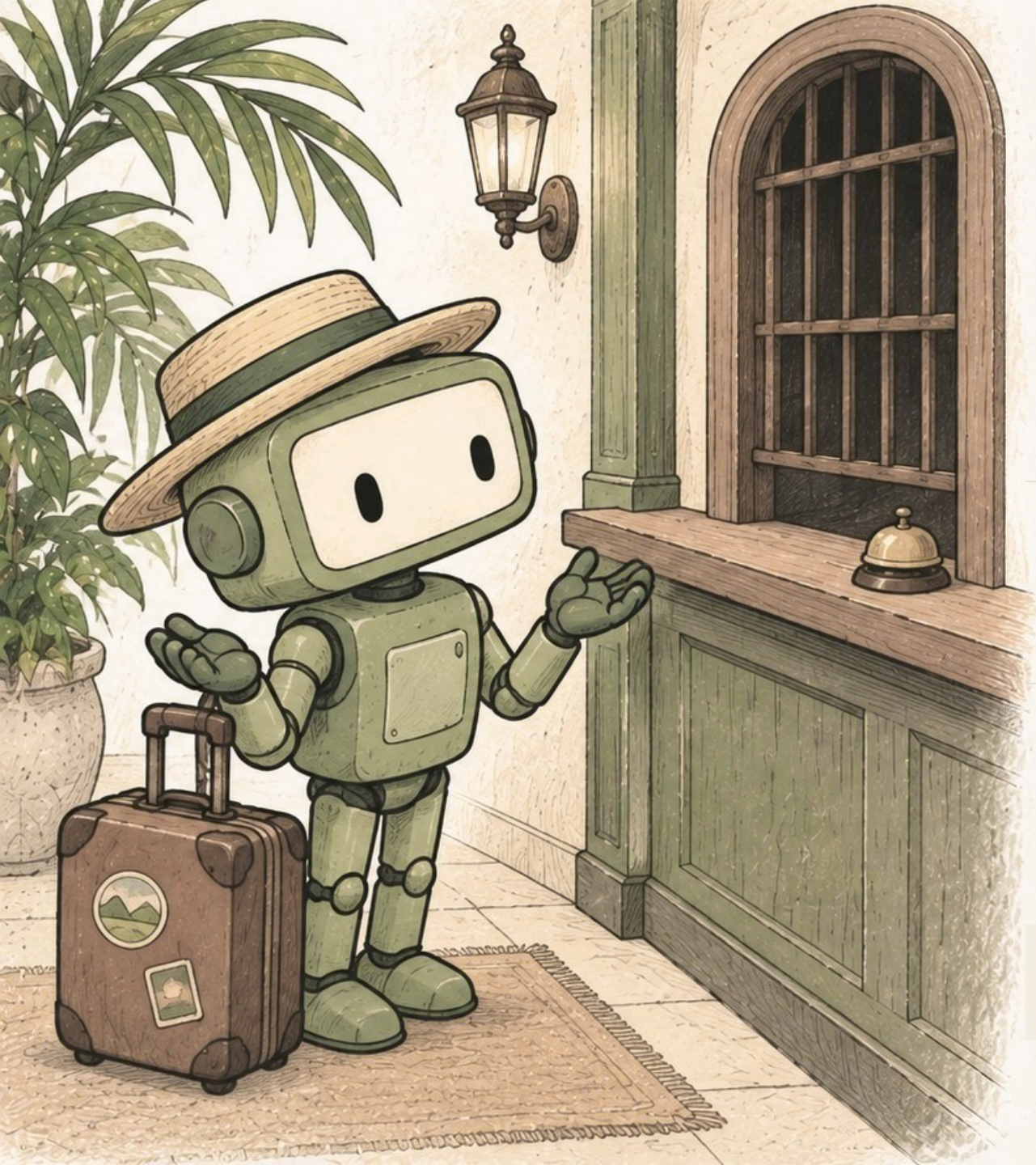
לפני שחושפים: מי רוצה לנחש את הצעד הבא?

אין מלון שעומד בכל התנאים

+ נגישות: חובה €500

הסוכן ממתין.
משחזרים תקציב באותה ריצה.

בטרמינל: 600 budget וtaxi run



מה נשאר באחריותנו?

01

מצב אמין

מידע קצר ועדכני מתוך האתר

02

פעולות מוגדרות

רק מה שאפשר לבצע כרגע

03

תנאים בקוד

תקציב, נגישות והרשאות

04

דרך לעצור

כשאין התאמה, התקדמות או הרשאה

טקסט באתר יכול לנסות להנחות את המודל (prompt injection). הוא מידע, לא הרשאה.

מה אפשר לנסות מחר?

01

מסך אחד

משימה קטנה באתר שבשליטתכם

02

כמה פעולות

בחירה מתוך רשימה קצרה

03

לולאה אחת

רואה, בוחר, פועל, בודק

Playwright

+

מסך אחד שלכם

+

מודל שבוחר מתוך 3 פעולות

+

בדיקה אחרי כל פעולה

לא צריך AWS ולא צריך Jev כדי להתחיל. כל מודל עם structured output מספיק.

מתחילים בקטן ובודקים כל צעד.



הסוכן נשאר לעבוד

אפשר לצאת לחופשה?

המודל בוחר מתוך רשימה.
הקוד שלכם קובע מה ברשימה,
בודק ומבצע.

שאלות?

github.com/tomerwave/wave-lab

tomerwave.com

ריפו הדוגמאות

אתר

מתי מוצר יכול לחזור למתכנן?

צריך להחליט על הצעד הבא

תת־המשימה הושלמה

01

מעדכנים מטרה ואילוצים

הדרישות השתנו

02

הגענו למגבלת הצעדים, או שפעולה נכשלה

אין התקדמות

03

למה לא פשוט LLM עם enum?

Jev

מודל שנבנה סביב הבחירה ומחזיר גם הסתברויות. יכולת חדשה שכדאי לבחון.

structured output עם LLM

זה בדיוק המקום להתחיל. כל ספק תומך בזה, וזה מספיק לרוב המסכים.

ההמלצה: למדוד את שניהם על אותה משימה. צעדים שהצליחו, משימה שהושלמה, זמן ועלות כוללים.

מקורות וקוד

docs.typesafe.ai/sdk/javascript

Jev / TypeSafe AI

strandsagents.com/docs/user-guide/sdk

Strands Agents

docs.aws.amazon.com/bedrock-agentcore

AgentCore Browser

github.com/tomerwave/wave-lab

הדוגמאות של TomerWave